Code

```
"""
PageRank with taxation (Section 5.1.5 of Mining of Massive Datasets).

Implements the iterative update:
        v' = beta * M * v + (1 - beta) * e / n

This handles spider traps by giving each random surfer a probability (1 - beta)
of teleporting to a random page on each step, instead of following an out-link.

We replicate Example 5.6 from page 187 using the graph of Fig. 5.6:
    A -> {B, C, D}
    B -> {A, D}
    C -> {C}              (one-node spider trap)
    D -> {B, C}

With beta = 0.8, the expected iterates are:
    [1/4, 1/4, 1/4, 1/4]
    [9/60, 13/60, 25/60, 13/60]
    [41/300, 53/300, 153/300, 53/300]
    [543/4500, 707/4500, 2543/4500, 707/4500]
    ...
    [15/148, 19/148, 95/148, 19/148]   (limit)
"""

from fractions import Fraction
import numpy as np



def pagerank_with_taxation(M, beta, num_iterations=None, tol=1e-12, max_iter=200):
    """
    Compute PageRank using the taxation formulation:
        v' = beta * M * v + (1 - beta) * e / n

    Parameters
    ----------
    M : (n, n) array-like
        Column-stochastic transition matrix. M[i, j] is the probability that
        a surfer at page j moves to page i in one step (i.e. 1/out_degree(j)
        if there is a link j -> i, else 0).
    beta : float
        Teleportation parameter. Typically 0.8 - 0.9. With probability beta the
        surfer follows an out-link; with probability (1 - beta) the surfer
        teleports to a random page.
    num_iterations : int or None
        If given, run exactly this many iterations and return the full history.
        If None, iterate until the L1 change between successive vectors is
        below `tol` (or `max_iter` is reached).
    tol : float
        Convergence threshold on the L1 norm of v' - v.
    max_iter : int
        Safety cap on iterations when num_iterations is None.

    Returns
    -------
    history : list of np.ndarray
        history[0] is the initial uniform vector v0 = (1/n, ..., 1/n) and
        history[k] is v after k iterations.
    """
    M = np.asarray(M, dtype=float)
    n = M.shape[0]

    # Start every surfer at a uniformly random page.
    v = np.ones(n) / n
    teleport = (1.0 - beta) * np.ones(n) / n   # the (1 - beta) * e / n term

    history = [v.copy()]

    if num_iterations is not None:
        for _ in range(num_iterations):
            v = beta * (M @ v) + teleport
            history.append(v.copy())
```

```
        return history

    for _ in range(max_iter):
        v_new = beta * (M @ v) + teleport
        history.append(v_new.copy())
        if np.sum(np.abs(v_new - v)) < tol:
            break
        v = v_new
    return history



def pagerank_exact(M_frac, beta_frac, num_iterations):
    """
    Same iteration but in exact rational arithmetic, so we can reproduce the
    textbook fractions like 9/60, 41/300, 543/4500 byte-for-byte.

    M_frac is a list of lists of Fractions; beta_frac is a Fraction.
    """
    n = len(M_frac)
    v = [Fraction(1, n)] * n
    one_minus_beta_over_n = (1 - beta_frac) / n

    history = [list(v)]
    for _ in range(num_iterations):
        Mv = [sum(M_frac[i][j] * v[j] for j in range(n)) for i in range(n)]
        v = [beta_frac * Mv[i] + one_minus_beta_over_n for i in range(n)]
        history.append(list(v))
    return history



# ---------------------------------------------------------------------------
# Reproduce Example 5.6 (Fig. 5.6 with a spider trap at C, beta = 0.8)
# ---------------------------------------------------------------------------

# Column j of M lists where a surfer at page j goes next.
# Pages are ordered A, B, C, D.
#   A: out-links to B, C, D       -> column A = [0, 1/3, 1/3, 1/3]
#   B: out-links to A, D          -> column B = [1/2, 0, 0, 1/2]
#   C: self-loop (spider trap)    -> column C = [0, 0, 1, 0]
#   D: out-links to B, C          -> column D = [0, 1/2, 1/2, 0]
M = [
    [0,         Fraction(1, 2), 0, 0           ],
    [Fraction(1, 3), 0,         0, Fraction(1, 2)],
    [Fraction(1, 3), 0,         1, Fraction(1, 2)],
    [Fraction(1, 3), Fraction(1, 2), 0, 0      ],
]
beta = Fraction(4, 5)   # 0.8
labels = ["A", "B", "C", "D"]



def fmt_fraction_vec(v):
    """Pretty-print a vector of Fractions with a common denominator."""
    from math import lcm
    denom = 1
    for x in v:
        denom = lcm(denom, x.denominator)
    parts = [f"{(x * denom).numerator}/{denom}" for x in v]
    return "[" + ", ".join(parts) + "]"



def main():
    print("=" * 72)
    print("PageRank with taxation - replicating Example 5.6 (beta = 0.8)")
    print("Graph: A->{B,C,D}, B->{A,D}, C->{C} (spider trap), D->{B,C}")
    print("=" * 72)

    # ---- Exact rational iteration to match the textbook's fractions --------
    print("\nExact iterations (rational arithmetic):\n")
    hist = pagerank_exact(M, beta, num_iterations=4)
    headers = ["iter 0 (v0)", "iter 1", "iter 2", "iter 3", "iter 4"]
    for k, v in enumerate(hist):
        print(f"  {headers[k]:<13}: {fmt_fraction_vec(v)}")

    # The textbook prints the limit as [15/148, 19/148, 95/148, 19/148].
```

```
    # Run many more iterations and confirm.
    hist_long = pagerank_exact(M, beta, num_iterations=200)
    limit = hist_long[-1]
    print("\nAfter 200 iterations (still exact rationals, rounded for display):")
    for lab, x in zip(labels, limit):
        print(f"  PR({lab}) = {float(x):.10f}")

    print("\nExpected textbook limit [15/148, 19/148, 95/148, 19/148]:")
    expected = [Fraction(15, 148), Fraction(19, 148),
                Fraction(95, 148), Fraction(19, 148)]
    for lab, x in zip(labels, expected):
        print(f"  PR({lab}) = {float(x):.10f}  ({x})")

    match = all(abs(float(a) - float(b)) < 1e-9 for a, b in zip(limit, expected))
    print(f"\nMatches textbook limit? {match}")

    # ---- Floating-point version (the implementation you would actually use)
    print("\n" + "=" * 72)
    print("Floating-point version, iterating until convergence (tol=1e-12):")
    print("=" * 72)
    M_float = [[float(x) for x in row] for row in M]
    hist_f = pagerank_with_taxation(M_float, beta=0.8, tol=1e-12, max_iter=500)
    final = hist_f[-1]
    print(f"\nConverged in {len(hist_f) - 1} iterations.")
    print("Final PageRank vector:")
    for lab, x in zip(labels, final):
        print(f"  PR({lab}) = {x:.10f}")
    print(f"\nSum of components: {final.sum():.10f}")


if __name__ == "__main__":
    main()
```

Results

Running the script produces the output below. The exact rational iterates match the textbook fraction-for-fraction:

```
======================================================================
PageRank with taxation - replicating Example 5.6 (beta = 0.8)
Graph: A->{B,C,D}, B->{A,D}, C->{C} (spider trap), D->{B,C}
======================================================================

Exact iterations (rational arithmetic):

  iter 0 (v0)  : [1/4, 1/4, 1/4, 1/4]
  iter 1       : [9/60, 13/60, 25/60, 13/60]
  iter 2       : [41/300, 53/300, 153/300, 53/300]
  iter 3       : [543/4500, 707/4500, 2543/4500, 707/4500]
  iter 4       : [2539/22500, 3263/22500, 13435/22500, 3263/22500]

After 200 iterations (still exact rationals, rounded for display):
  PR(A) = 0.1013513514
  PR(B) = 0.1283783784
  PR(C) = 0.6418918919
  PR(D) = 0.1283783784

Expected textbook limit [15/148, 19/148, 95/148, 19/148]:
  PR(A) = 0.1013513514  (15/148)
  PR(B) = 0.1283783784  (19/148)
  PR(C) = 0.6418918919  (95/148)
  PR(D) = 0.1283783784  (19/148)

Matches textbook limit? True

======================================================================
Floating-point version, iterating until convergence (tol=1e-12):
======================================================================
```

```
Converged in 51 iterations.
Final PageRank vector:
  PR(A) = 0.1013513514
  PR(B) = 0.1283783784
  PR(C) = 0.6418918919
  PR(D) = 0.1283783784

Sum of components: 1.0000000000
```

Verification

Iterates 0 through 3 reproduce the four vectors printed in Example 5.6 exactly: [1/4, 1/4, 1/4, 1/4], [9/60, 13/60, 25/60, 13/60], [41/300, 53/300, 153/300, 53/300], and [543/4500, 707/4500, 2543/4500, 707/4500]. After enough iterations the rational version converges to [15/148, 19/148, 95/148, 19/148], which is the limiting vector the textbook reports. The floating-point implementation converges to the same values (to 10 decimal places) in 51 iterations with a tolerance of 1e-12.

Note how the spider trap at C still attracts the largest share of PageRank (about 0.642), but taxation prevents it from absorbing *all* of it the way the un-taxed iteration in Example 5.5 did (where C ended up with PageRank 1 and the other three pages with 0). Every page now retains a positive PageRank.